



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **informatyka**

Mateusz Kowal

nr albumu: 318683

Środowisko programowalnego bezzałogowego statku powietrznego z użyciem mikrokomputera Raspberry Pi

A programmable unmanned aerial vehicle environment
based on a Raspberry Pi microcomputer

Praca licencjacka

napisana w Katedrze Oprogramowania Systemów Informatycznych

Instytutu Informatyki i Matematyki UMCS

pod kierunkiem **dr hab. Beaty Byliny**

Lublin 2026

Spis treści

Wstęp	5
1 Wprowadzenie do bezzałogowych statków powietrznych	7
1.1 Podstawowe pojęcia	7
1.2 Zadania i ograniczenia komputera pokładowego	8
1.3 Rozszerzenie możliwości BSP dzięki komputerom asystującym	8
2 Elementy sprzętowe BSP	11
2.1 Lista komponentów i ich specyfikacja	11
2.1.1 Rama BSP	11
2.1.2 Napęd	11
2.1.3 Zewnętrzny moduł nawigacji satelitarnej	14
2.1.4 Układ komunikacji z pilotem	14
2.1.5 Komputer pokładowy	14
2.1.6 Komputer asystujący	15
2.1.7 Źródło zasilania	15
2.1.8 Montaż komputerów	15
2.2 Specyfikacja drona	17
3 Zastosowanie środowiska	19
3.1 Wstępne założenia	19
3.2 Projekt urządzenia peryferyjnego— system czujników odległości	22
3.3 Program wykrywający przeszkody	23
3.4 Prędkość jako jeden z warunków aktywacji	27
3.5 Dynamiczne ustawianie progu odległości	30
3.6 Testy	33
3.6.1 Sprawdzenie poprawności działania systemu	33
3.6.2 Pobór prądu	34
3.6.3 Dokładność pomiarów odległości	34
3.6.4 Czas reakcji systemu	35

3.7	Alternatywne rozwiązania	36
4	Potencjalne ścieżki rozwoju projektu	39
4.1	Półautonomiczny system zapobiegania kolizjom	39
4.2	Wielokierunkowy monitoring	39
4.3	Sieć neuronowa rozpoznawanie obiektów z obrazu kamery	39
4.4	mapa 3d lidar	39
	Bibliografia	39

Wstęp

Tu treść wstępu (który piszemy na końcu, po napisaniu całej pracy).

Rozdział 1

Wprowadzenie do bezzałogowych statków powietrznych

1.1 Podstawowe pojęcia

Bezzałogowy statek powietrzny (skrót BSP) to kategoria pojazdów poruszających się w powietrzu do której zaliczane są samoloty, śmigłowce i pojazdy wielowirnikowe (w tym drony), które odbywają loty bez pilota na pokładzie. Sterowanie może się odbywać zdalnie przez pilota naziemnego lub przez komputer, jeśli pojazd jest autonomiczny [2].

Komputer pokładowy jest urządzeniem niezbędnym do działania BSP. Jego głównym zadaniem jest sterowanie silnikami pojazdu oraz otrzymywanie i przetwarzanie sygnałów docierających z układu sterowania. W tym przypadku rolę komputera pokładowego pełni Pixhawk 6C.

Komputer asystujący to komputer znajdujący się na pokładzie BSP, połączony z komputerem pokładowym. Jego rolą jest wykonywanie zadań zbyt skomplikowanych obliczeniowo by móc powierzyć je komputerowi pokładowemu lub wymagają elementów przez komputer pokładowy nieposiadanych i niewspieranych. Komputerem asystującym w przypadku tego projektu jest Raspberry Pi 5B.

Globalny System Nawigacji Satelitarnej (ang *Global Navigation Satellite System*, skrót GNSS) to system pozwalający na pozycjonowanie i znajdowanie ścieżki używając do tego sztucznych satelitów na orbicie Ziemi. Potocznie występuje pod nazwą GPS, w rzeczywistości jest to amerykańska sieć satelitów i jest jedną z kilku systemów składających się na GNSS.

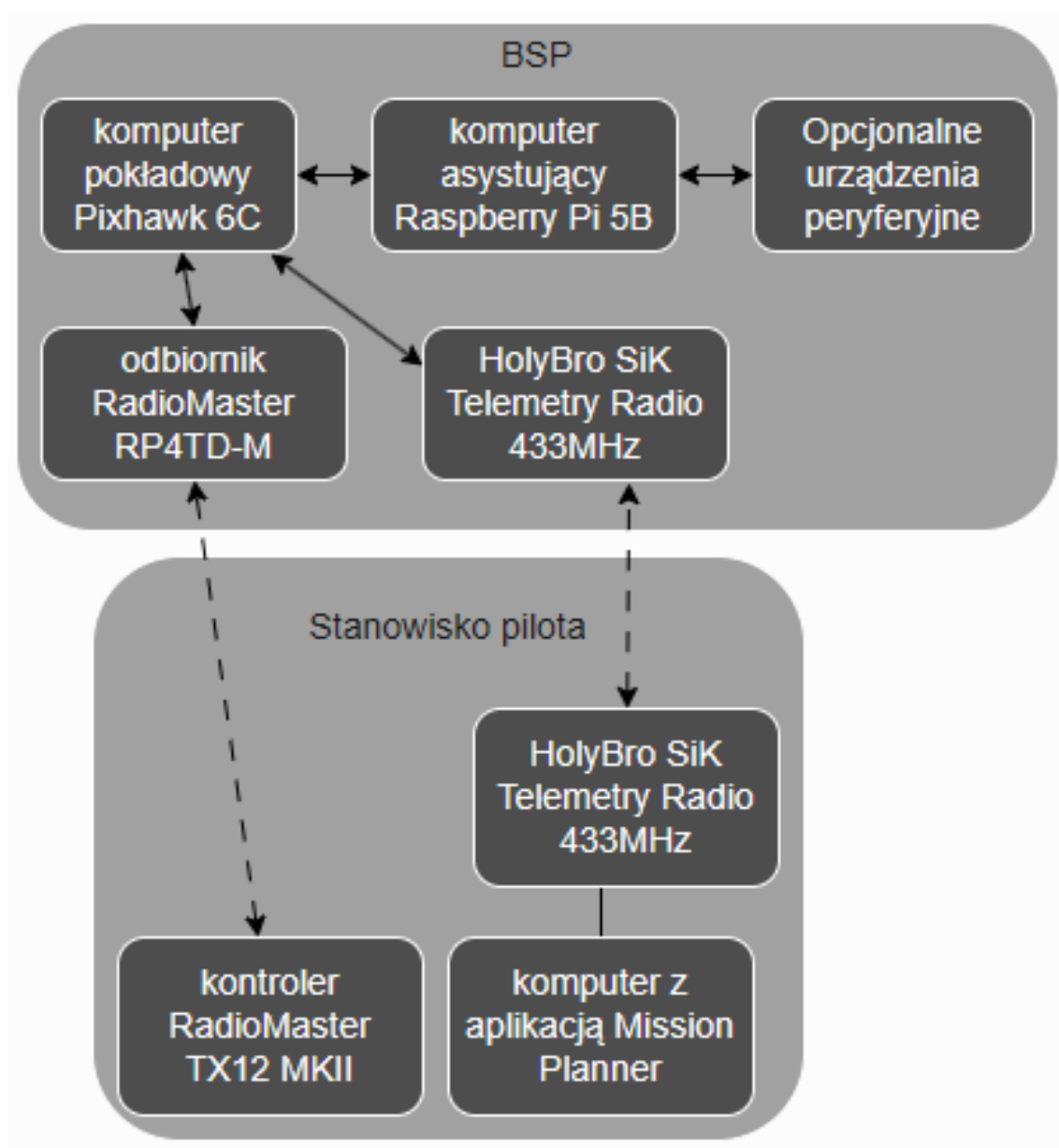
1.2 Zadania i ograniczenia komputera pokładowego

Rolą każdego komputera pokładowego jest przede wszystkim sterowanie silnikami pojazdu którego jest częścią. Komputer pokładowy musi więc posiadać odpowiednie interfejsy do komunikacji z silnikami oraz interfejs do otrzymywania poleceń. W przypadku komputerów pokładowych używanych w BSP wielowirnikowych niezbędny jest również żyroskop, dzięki któremu komputer pokładowy zna swoją pozycję i jest w stanie utrzymać pojazd poziomo lub odpowiednio odchylić go zgodnie z poleceniami które otrzyma.

W zależności od modelu komputera pokładowego oraz zainstalowanego na nim oprogramowania może on dodatkowo obsługiwać wiele urządzeń peryferyjnych i posługiwać się bardziej złożoną logiką. Przeciętny komputer pokładowy dostępny na rynku cywilnym oferuje możliwość podłączenia zewnętrznego modułu GPS czy kamery, a także samodzielnie przechowywać i wykonywać polecenia pozwalające na lot autonomiczny. Standardem jest również możliwość ustawienia procedury bezpieczeństwa (*ang. failsafe*), która decyduje jak zachowa się pojazd w razie utraty podłączenia z kontrolerem naziemnym lub jakimkolwiek innym urządzeniem wydającym polecenia komputerowi pokładowemu.

1.3 Rozszerzenie możliwości BSP dzięki komputerom asystującym

Jeśli użytkownik chce wykorzystać swój BSP w bardziej nietypowy sposób, na drodze mogą stać mu ograniczenia sprzętowe. Komputery pokładowe oferują zazwyczaj niewielką moc obliczeniową i ograniczoną liczbę i różnorodność interfejsów do podłączenia urządzeń peryferyjnych. Jednym ze sposobów przystosowania BSP do naszych wymagań jest użycie komputera asystującego który rozszerzy możliwości jednostki. Przykładowo, użytkownik może chcieć użyć swojego BSP do monitorowania dużego obszaru. W tym celu chciałby zamontować kamery z wielu stron, jednak użyty przez niego komputer pokładowy ma możliwość podpięcia tylko jednej kamery. Użycie komputera asystującego może być wtedy rozwiązaniem problemu. W zależności od użytego komputera asystującego można użyć nawet tak złożonych obliczeniowo programów jak sztuczna sieć neuronowa analizująca dane z urządzeń wejściowych i wydająca polecenia jednostce. Sposób połączenia komputera asystującego oraz połączeń dających użytkownikowi kontrolę nad dronem i informacje zwrotne zostały przedstawione na rysunku [1.1](#).



Rysunek 1.1: Schemat wybranych komponentów jednostki i połączenia między nimi

Rozdział 2

Elementy sprzętowe BSP

2.1 Lista komponentów i ich specyfikacja

Większość gotowych bezzałogowych pojazdów dostępnych na rynku to zamknięte systemy a ingerencje i modyfikacje są bardzo utrudnione, zarówno sprzętowo jak i w oprogramowaniu. Samodzielna budowa jednostki z użyciem kupionych oddzielnie komponentów oraz oprogramowania open-source daje dużo większe możliwości w przystosowaniu platformy do własnych wymagań i preferencji.

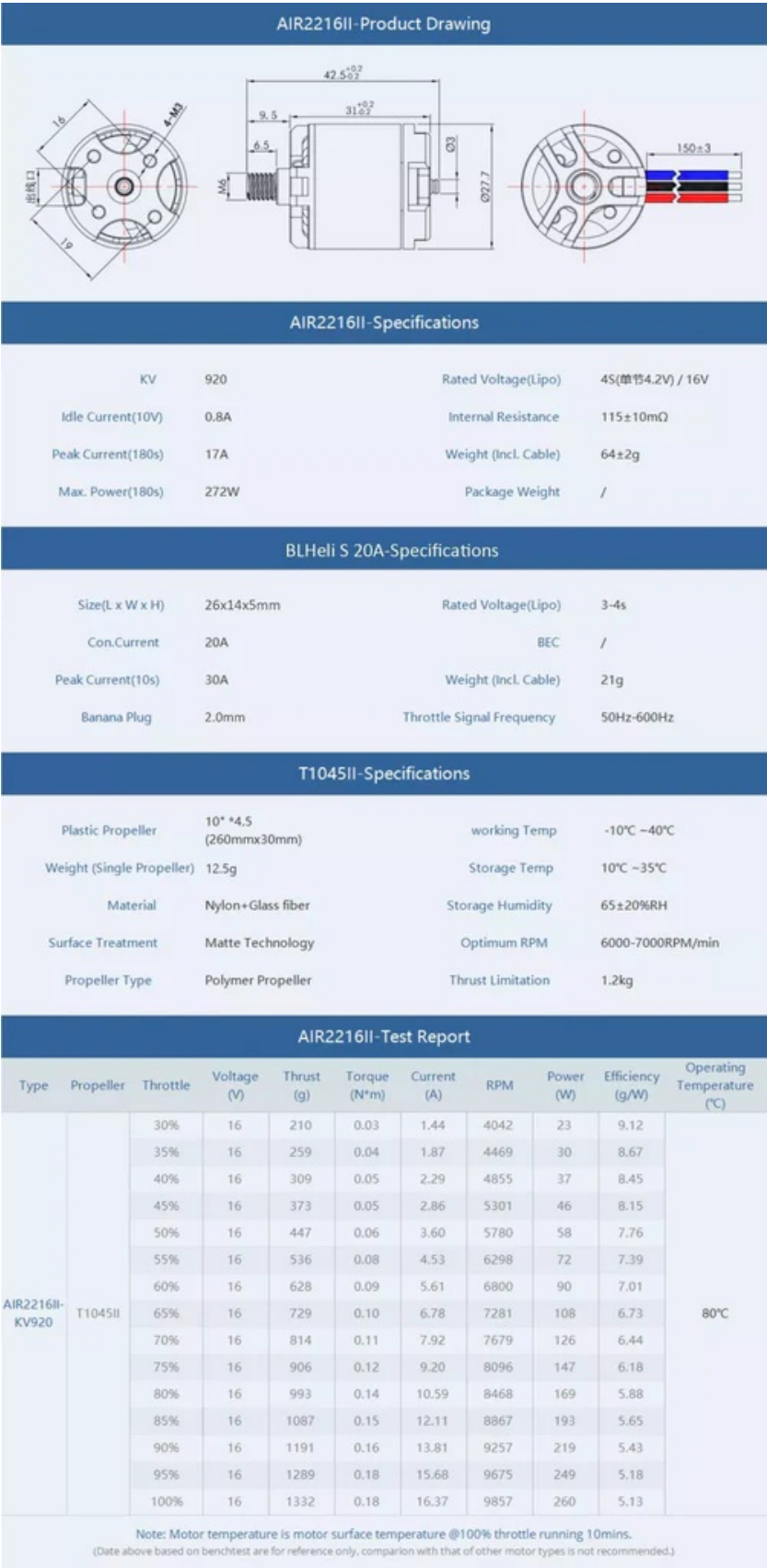
2.1.1 Rama BSP

Rama użyta do zbudowania jednostki to Holybro S500 v2. Oferuje wbudowaną w konstrukcję kadłuba rozdzielnicę napięcia (*ang. power distribution board*) dostarczającą energię elektryczną z akumulatora do silników. Miejscem na montaż silników są cztery ramiona wykonane z nylonu poliamidowego, wzmocnione w środku poprzez dodanie prętów z włókna węglowego. Całość opiera się na podwoziu z rurek z włókna węglowego połączonych plastikowymi łącznikami.

2.1.2 Napęd

Pojazd napędzany jest przez cztery silniki bezszczotkowe AIR2216II o mocy do 272 W każdy. Silniki bezszczotkowe wymagają zasilania prądem zmiennym trójfazowym, dlatego do ich działania potrzebne są moduły regulacji napięcia (*ang. electronic speed controller*). Moduły te zasilane są prądem stałym z akumulatora, który następnie zostaje przekształcony na prąd zmienny trójfazowy wywołujący obrót silników o zadanej prędkości. Prędkość jest regulowana za pomocą sygnału dochodzącego do ESC z komputera pokładowego. Moduły ESC zastosowane w zbudowanym pojeździe to BLHeli S 20A. Jednostka została wyposażona w śmigła T1045II o rozpiętości 260

milimetrów i szerokości do 30 milimetrów. Specyfikacja każdego z elementów napędu została pokazana na rysunku [2.1](#).



Rysunek 2.1: Dokumentacja techniczna AIR2216II, BLHeli S 20A, T1045II. Źródło: [3]

2.1.3 Zewnętrzny moduł nawigacji satelitarnej

Zastosowany w jednostce zewnętrzny moduł GNSS to M10 od firmy Holybro. Oferuje jednocześnie otrzymywanie informacji od czterech największych systemów satelit geolokacyjnych: europejskich satelit Galileo, amerykańskich GPS, rosyjskich GLO-NASS i chińskich Bei Dou. Oferowana dokładność wyznaczonej pozycji to 2.0 m CEP, co oznacza że co najmniej 50% pomiarów wskaże pozycję oddaloną o 2 metry lub mniej od rzeczywistej pozycji [4].

2.1.4 Układ komunikacji z pilotem

BSP został przygotowany do utrzymywania dwóch rodzajów komunikacji z pilotem. Podstawowym kanałem komunikacji jest kontroler RadioMaster TX12 MKII pozwalający na zdalne sterowanie dronem wyposażonym w odbiornik radiowy RadioMaster RP4TD-M używając systemu ExpressLRS zapewniającego niezawodność i stabilność łącza nawet na dystansie 100 km [5]. Drugim kanałem komunikacji jest system telemetrii Holybro SiK Telemetry Radio V3 w wersji 433 MHz. Moduł podłączony do komputera osobistego znajdującego się na ziemi w pobliżu pilota pozwala na użycie aplikacji Mission Planner—oprogramowania pozwalającego między innymi na śledzenie BSP na mapie czy odczyt bieżących danych [1].

2.1.5 Komputer pokładowy

Rolę komputera pokładowego pełni Pixhawk 6C. Wyposażony jest w dwa procesory: STM32H743 — 480MHz Cortex-M7 odpowiedzialny za działanie oprogramowania zainstalowanego na urządzeniu, odczyt parametrów z sensorów, przetwarzanie informacji i przekazywanie poleceń dotyczących silników do drugiego procesora, w tym przypadku STM32F103 — 72 MHz Cortex-M3. Zadaniem drugiego procesora jest generowanie sygnałów modulacji szerokości pulsu (ang *pulse-width modulation*) na podstawie otrzymanych danych i przekazywanie ich do modułów ESC. W przypadku wystąpienia problemów z pierwszym procesorem jest w stanie samodzielnie generować sygnał sterujący silnikami, jednak przez brak dostępu do czujników takich jak żyroskopy czy akcelerometry jego możliwości ograniczają się do utrzymania kręcenia się śmigieł w celu spowolnienia upadku i minimalizacji uszkodzeń przy lądowaniu awaryjnym.

Do poprawnego sterowania pojazdem wymagane są również odpowiednie czujniki. Pixhawk 6C posiada dwa urządzenia łączące w sobie akcelerometry i żyroskopy: ICM-42688-P i BMI055. Użycie dwóch urządzeń i porównanie ich odczytów ze sobą pozwala na wykrycie niedokładności lub uszkodzenia jednego z nich i zapobiega destabilizacji drona w przypadku awarii. W razie uszkodzenia komputer pokładowy

może wdrożyć procedurę bezpieczeństwa i bezpiecznie ją wykonać polegając na pozostałym działającym czujniku. Pixhawk 6C wyposażony jest również w magnetometr IST8310 pozwalający na określenie jego pozycji na podstawie pola magnetycznego Ziemi oraz barometr MS5611 pozwalający obliczyć wysokość nad ziemią na podstawie różnicy ciśnień [8]. Oprogramowaniem działającym na komputerze pokładowym jest ArduCopter w wersji 4.6.2.

2.1.6 Komputer asystujący

Asystentem dla Pixhawk 6C jest mikrokomputer Raspberry Pi 5B. Posiada on czterordzeniowy procesor Arm Cortex-A76, 8 GB pamięci RAM oraz szereg interfejsów pozwalających na podłączenie urządzeń peryferyjnych, np. cztery gniazda USB czy 40 pinów GPIO. Zainstalowany na nim system operacyjny to Raspberry Pi OS będący portem Debiana Trixie.

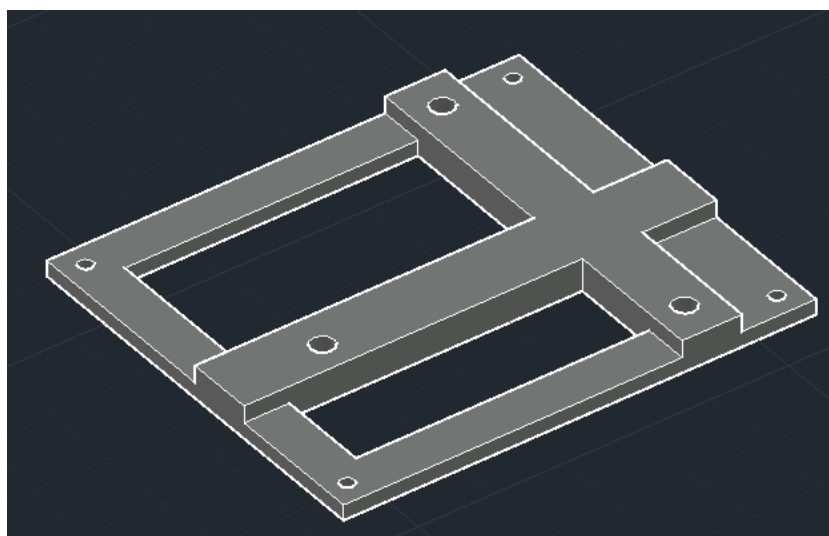
2.1.7 Źródło zasilania

BSP zasilany jest za pomocą akumulatora litowo-jonowego Gens Ace G-Tech o pojemności 5000 mAh, napięciu 14.8 V i wadze 439 g. Za zasilanie komputera asystującego odpowiedzialny jest powerbank z baterią litowo-polimerową o pojemności 10000 mAh, napięciu 5 V, mocy maksymalnej 22.5 W i wadze 235 g. Możliwe jest zasilanie Raspberry Pi akumulatorem drona, wiąże się to jednak z ingerencją w gotowy układ zasilania oraz koniecznością obniżenia napięcia z 14.8 V do 5 V. Problem stanowią również silniki elektryczne napędzające jednostkę, ich zużycie prądu zmienia się bardzo dynamicznie co stwarza ryzyko spadku napięcia zasilającego komputer asystujący poniżej wymaganych 5 V. W przypadku Raspberry Pi spadek napięcia poniżej 4.6 3V ($\pm 5\%$) może skutkować obniżoną wydajnością komputera, nieprawidłowym działaniem podłączonych urządzeń peryferyjnych lub do nieprzewidywalnego zachowania urządzenia czy uszkodzenia danych na karcie pamięci. [6] Aby uniknąć potencjalnych problemów i ingerencji w gotowy układ zasilania zastosowane zostało oddzielne źródło prądu dla komputera asystującego.

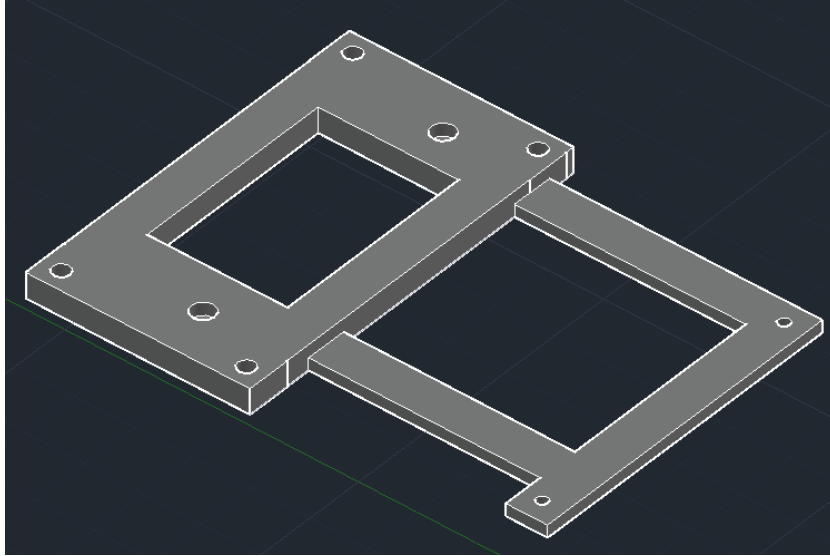
2.1.8 Montaż komputerów

Kontroler lotu Pixhawk 6C ma w zestawie dwustronną taśmę samoprzylepną pozwalającą na łatwy montaż komputera. Taśma ma dodatkowo warstwę pianki pomiędzy warstwami przyklepnymi w celu amortyzacji wstrząsów i ochrony komputera przed uszkodzeniami mechanicznymi. Rozmiary kadłuba drona nie pozwalają na umieszczenie obu komputerów obok siebie a montaż jeden na drugim nie jest możliwe z użyciem

taśmy. Z tego powodu zastąpiono ją elementami zaprojektowanymi w programie AutoCAD i wydrukowanymi na drukarce 3D. Rama montażowa dla Raspberry Pi widoczna jest na rysunku 2.2. Kształt uchwytu jest dopasowany tak, aby dało się go przymocować do kadłuba pomimo wystających z niego śrub. Do montażu ramy do BSP oraz mikrokomputera do ramy użyto śrub M2. Pomiedzy uchwytem a Raspberry Pi dodano piankę amortyzującą wstrząsy a mikrokomputer został przymocowany z użyciem mosiężnych sześciokątnych kolumn o długości 2 cm i gwintowanych otworach. Na kolumny została przykręcona druga rama 2.3 pozwalająca na montaż komputera pokładowego Pixhawk. Została ona zaprojektowana tak, by pozostawić odsłonięte chłodzenie Raspberry Pi i piny GPIO.



Rysunek 2.2: Uchwyt montażowy dla Raspberry Pi widoczny od dołu



Rysunek 2.3: Uchwyt montażowy dla Pixhawk 6C

2.2 Specyfikacja drona

Waga jednostki, wliczając w to komputer asystujący i jego źródło zasilania, wynosi 1715 g. Na podstawie wagi, specyfikacji silników [3] oraz akumulatora można oszacować średni możliwy czas lotu. Aby dron zawisł w bezruchu, każdy z czterech silników musi wytworzyć ok. 428.75 g ciągu. Dzięki dokumentacji wiadomo że jeden silnik z zastosowanym dołączonym w zestawie śmigłem potrzebuje 2.86 A do wytworzenia 373 g ciągu i 3.60 A dla 447 g ciągu. Prąd potrzebny do wytworzenia dokładnie 428.75 g ciągu może być obliczony z użyciem interpolacji liniowej:

$$I = I_1 + \frac{T - T_1}{T_2 - T_1}(I_2 - I_1),$$

$$I = 2.86 + \frac{428.75 - 373}{447 - 373}(3.60 - 2.86),$$

$$I \approx 3.42 \text{ A}.$$

Jeden silnik potrzebuje około 3.42 A, co dla czterech silników daje łącznie 13.67 A. Używając akumulatora o pojemności 5000 mAh daje to teoretyczne 21.95 minut lotu. Biorąc pod uwagę dodatkowe zużycie prądu przez komputer pokładowy oraz zwiększone zużycie prądu przy wznoszeniu i manewrowaniu, bezpieczniejszym będzie przyjęcie marginesu błędu i określenie czasu lotu na około 19–20 minut. Oszacowanie maksymalnego czasu działania Raspberry Pi jest dużo trudniejsze, zależy to w dużym stopniu od złożoności programów na niej uruchomionych. Opisane dalej testy wykazały że pobór

mocy w stanie spoczynku wahają się między 3.177 W a 3.484 W, mediana odczytów to 3.281 W. Przy parametrach zastosowanego zasilania szacowany czas działania to 11 godzin i 15 minut.

Rozdział 3

Zastosowanie środowiska

3.1 Wstępne założenia

Komputer Raspberry Pi jest połączony z Pixhawk 6C za pomocą USB. W katalogu `~/uavproject` znajduje się wirtualne środowisko python oraz wszystkie programy współpracujące z dronem. Komunikacja z dronem możliwa jest dzięki bibliotekom DroneKit 2.9.2 i PyMAVLink 2.4.49. Użycie tych bibliotek ogranicza wersję pythona do nie nowszej niż 3.9.18, ponieważ od jakiegoś czasu biblioteki są nieaktualizowane i nie współpracują z nowszymi wersjami pythona. Za pomocą komendy `source uavproject/venv/bin/activate` uruchamiane jest wirtualne środowisko. Wewnątrz środowiska można testować napisane programy i instalować wymagane biblioteki. Pisanie programu najlepiej zacząć używając szablonu 3.1.

Listing 3.1: Szablon programu działającego na komputerze asystującym

```
1 from dronekit import connect, APIException
2 from pymavlink import mavutil
3 import os
4 import time
5
6 DRONE_PORT = """/dev/serial/by-id/usb-Holybro_Pixhawk6C_
7             450046001751333337363133-if00""
8
9 def connect_pixhawk():
10     print("Connecting to Pixhawk...")
11
12     while True:
13         try:
14             vehicle = connect(
15                 DRONE_PORT,
16                 baud=115200,
17                 wait_ready=True,
```

```
18         heartbeat_timeout=60
19     )
20     print("Connected to Pixhawk")
21     return vehicle
22
23     except APIException as e:
24         print(f"No heartbeat yet: {e}")
25         time.sleep(3)
26
27     except Exception as e:
28         print(f"Connection failed: {e}")
29         time.sleep(3)
30 def main():
31     vehicle = None
32
33     while not os.path.exists(DRONE_PORT):
34         time.sleep(1)
35     try:
36         vehicle = connect_pixhawk()
37
38         vehicle.message_factory.statustext_send(
39             mavutil.mavlink.MAV_SEVERITY_ALERT,
40             b"Companion computer connected"
41         )
42         vehicle.flush()
43
44         #miejsce na kod
45
46     except KeyboardInterrupt:
47         print("\nProgram interrupted by user.")
48
49     finally:
50         print("Closing connections...")
51
52         if vehicle:
53             vehicle.close()
54             print("Drone connection closed")
55
56 if __name__ == "__main__":
57     main()
58
```

Szablon obejmuje poprawne otwarcie połączenia z komputerem pokładowym za pomocą funkcji `connect_pixhawk()`. Funkcja tworzy połączenie i nasłuchuje sygnału *heartbeat*. Sygnał ten jest wysyłany przez komputer pokładowy w momencie gdy ten

jest uruchomiony i gotowy do pracy. Po otrzymaniu sygnału połączenie jest zwracane i może być użyte do komunikacji z dronem w dalszej części programu. Program, używając otwartego połączenia, wywołuje wysłanie wiadomości przez komputer Pixhawk o pomyślnym podłączeniu komputera asystującego. Wiadomość może być odczytana przez pilota mającego połączenie z jednostką dzięki aplikacji Mission Planner. Szablon uwzględnia też poprawne zamknięcie połączenia, również w przypadku jeśli działanie programu zakończy się poprzez przerwanie skrótem klawiszowym lub błąd programu. Bez zamknięcia połączenia każda kolejna próba uruchomienia programu zakończy się błędem, ponieważ urządzenie bez poprawnie zamkniętego połączenia będzie oznaczone jako zajęte i niedostępne.

Loty dronem najczęściej nie odbywają się w warunkach domowych, dlatego programy powinny się wykonywać automatycznie, bez potrzeby użycia klawiatury i monitora. W tym celu wykorzystany został skrypt 3.2 zapisany w `/etc/systemd/system/autorunuav.service`. Po uruchomieniu Raspberry Pi skrypt poczeka aż system będzie w pełni uruchomiony, po czym wykona program określony w parametrze `ExecStart` przy użyciu wirtualnego środowiska. W przypadku jeśli program przestanie działać skrypt odczeka pięć sekund i uruchomi go ponownie. Po każdej zmianie automatycznie uruchamianego programu lub jakichkolwiek innych zmianach w skrypcie należy odświeżyć systemd komendą `sudo systemctl daemon-reload` oraz uruchomić skrypt za pomocą `sudo systemctl start autorunuav.service`. Wyjście programu można zobaczyć używając komendy `journalctl -u autorunuav.service -f`.

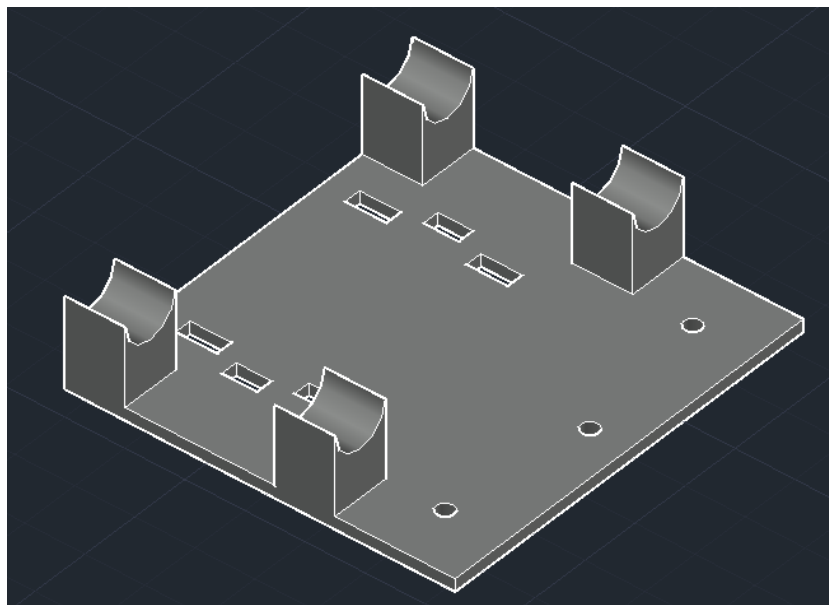
Listing 3.2: Skrypt wykonujący program po uruchomieniu komputera asystującego

```
1 [Unit]
2 Description=UAV Companion Autostart
3 After=multi-user.target
4
5 [Service]
6 Type=simple
7 User=uavcompanion
8 Group=uavcompanion
9
10 WorkingDirectory=/home/uavcompanion/uavproject
11 ExecStart=/home/uavcompanion/uavproject/venv/bin/python -u program.py
12
13 Restart=on-failure
14 RestartSec=5
15
16 StandardOutput=journal
17 StandardError=journal
```

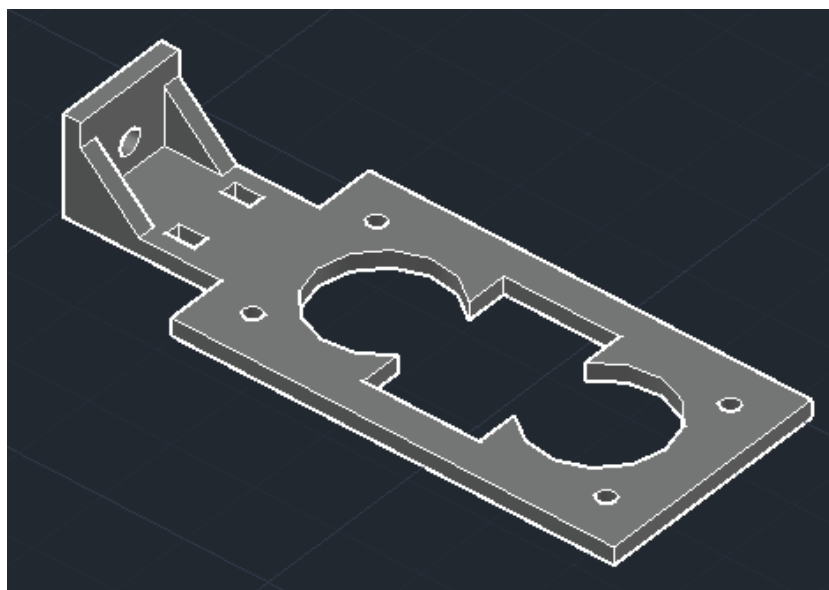
```
18  
19 [Install]  
20 WantedBy=multi-user.target  
21
```

3.2 Projekt urządzenia peryferyjnego— system czujników odległości

W celu przetestowania zbudowanej platformy i zaprezentowania jednej z możliwości jej wykorzystania zbudowano system ostrzegania przed kolizją. Wykrywanie przeszkód odbywa się z użyciem trzech ultradźwiękowych czujników odległości HC-SR04 [9]. Zasilanie 5 V oraz uziemienie są połączone równolegle do pinów w Raspberry Pi. Piny TRIG w czujnikach, wywołujące pomiar, są podłączone bezpośrednio do pinów GPIO. Wyjście ECHO czujników nie może być podłączone do Raspberry Pi ze względu na różnicę w napięciach. W czujnikach HC-SR04 dla sygnału wychodzącego przez pin ECHO stan wysoki to 5 V. Takie napięcie jest odpowiednie dla mikrokomputera Arduino, jednak stan wysoki dla pinów w Raspberry Pi to 3.3 V. Aby zapobiec uszkodzeniom należy obniżyć napięcie wychodzące z czujnika do bezpiecznych 3.3 V, co zostało osiągnięte z użyciem konwertera poziomów logicznych. Podobnie jak w przypadku komputerów, montaż konwertera stanów logicznych i czujników zostały zaprojektowane i wydrukowane w 3D. Każdy z trzech czujników został zamontowany za pomocą pionowych uchwytów widocznych na rysunku 3.2, które zostały przykręcone do platformy pokazanej na rysunku 3.1. Platforma ma również miejsce na zamocowanie konwertera stanów logicznych. Całość została podwieszona z użyciem opasek zaciskowych na dwóch rurkach z włókna węglowego będących częścią kadłuba drona.



Rysunek 3.1: Rama montażowa do zamocowania konwertera stanów logicznych i uchwytów na czujniki



Rysunek 3.2: Uchwyt na jeden czujnik ultradźwiękowy

3.3 Program wykrywający przeszkody

Po zbudowaniu poprawnie działającego systemu czujników odległości opracowano program [3.3](#) wykorzystujący je do ostrzegania pilota przed przeszkodami znajdującymi się przed dronem. Na początku programu znajdują się zmienne globalne pozwalające na

łatwą zmianę kluczowych parametrów, takich jak odległość na której system powinien zareagować lub maksymalną odległość mierzoną przez czujniki. Następnym krokiem jest utworzenie połączenia z komputerem pokładowym w sposób ukazany wcześniej w szablonie 3.1. Po pomyślnym połączeniu program wchodzi w nieskończoną pętlę, w której wykonuje pomiary za pomocą kolejnych czujników i porównuje odczytane odległości do progu ustalonego w zmiennej globalnej `THRESHOLD`. Jeżeli przeszkoda wykryta przez czujnik okaże się bliżej niż próg, program wyśle polecenie do komputera pokładowego po którym ten wyśle powiadomienie do pilota. Wysyłanie powiadomienia działa w ten sam sposób co powiadomienie wysyłane po pomyślnym otwarciu połączenia z Pixhawk. Wartości zmiennych globalnych `MAX_DIST` oraz `GAP` zostały ustalone dzięki testom opisanym w sekcji 3.5.3. Wartość `GAP` to odstęp w czasie pomiędzy kolejnymi odczytami. W przypadku użycia wielu czujników obok siebie istnieje ryzyko że po wywołaniu pomiaru u jednego z czujników, przejdziemy do drugiego zanim poprzedni zakończy pomiar. W takiej sytuacji może się zdarzyć że fala ultradźwiękowa wysłana przez poprzedni czujnik dotrze do kolejnego, co skutkować będzie zaburzonym pomiarem i fałszywie krótkim zmierzonym dystansem. Ryzyko wystąpienia takiego problemu jest największe gdy czujniki zwrócone są w tę samą stronę, w przypadku zbudowanego systemu ostrzegania ryzyko zaburzenia pomiaru z tego powodu jest znikome. Mimo wszystko pozostawienie odstępu jest wskazane, a odstęp 27 ms nie jest wystarczająco długi aby uczynić program zawodnym przez zbyt rzadkie pomiary. Każda wartość pomiaru zwracana jest w postaci liczby zmiennoprzecinkowej pomiędzy 0 a 1, gdzie 0 oznacza 0 cm a 1 oznacza maksymalną odległość pomiaru określaną w zmiennej `MAX_DIST`. Wartość pomiaru jest mnożona przez 100 i maksymalną odległość pomiaru w celu uzyskania wartości w centymetrach.

Listing 3.3: Program ostrzegający o wykrytych przez czujniki przeszkodach

```
1 import time
2 import math
3 from dronekit import connect
4 from pymavlink import mavutil
5
6 from gpiozero import Device, DistanceSensor
7 from gpiozero.pins.lgpio import LGPIOPinFactory
8
9 Device.pin_factory = LGPIOPinFactory()
10
11 DRONE_PORT = "/dev/serial/by-id/usb-Holybro_Pixhawk6C_450046001751333337363133-if0
12
13 MIN_DISTANCE_CM = 30
14 MIN_SPEED = 0.3 # w m/s
15 GAP = 0.027 # 27ms
```



```
16 MAX_DIST = 4.5
17
18 SENSOR_ANGLES_DEG = {
19     1: -45.0,
20     2: 0.0,
21     3: 45.0
22 }
23
24 sensor1 = DistanceSensor(echo=17, trigger=23, max_distance=MAX_DIST)
25 sensor2 = DistanceSensor(echo=27, trigger=24, max_distance=MAX_DIST)
26 sensor3 = DistanceSensor(echo=22, trigger=25, max_distance=MAX_DIST)
27
28
29 def connect_pixhawk():
30     #[...]
31
32
33 def get_body_velocity(vehicle):
34     velocity = vehicle.velocity
35     yaw = vehicle.attitude.yaw
36
37     if velocity is None or yaw is None:
38         return 0.0, 0.0
39
40     vn, ve, _ = velocity
41
42     v_forward = vn * math.cos(yaw) + ve * math.sin(yaw)
43     v_right = -vn * math.sin(yaw) + ve * math.cos(yaw)
44
45     return v_forward, v_right
46
47
48 def get_speed_toward_sensor(v_forward, v_right, sensor_angle_deg):
49     angle_rad = math.radians(sensor_angle_deg)
50     return v_forward * math.cos(angle_rad) + v_right * math.sin(angle_rad)
51
52
53 def send_alert(vehicle):
54     message = "obstacle"
55
56     vehicle.message_factory.statustext_send(
57         mavutil.mavlink.MAV_SEVERITY_ALERT,
58         message.encode("utf-8")
59     )
60     vehicle.flush()
```

```
61
62
63 def check_distance(vehicle, distance_cm, sensor_id, v_forward, v_right):
64     sensor_angle = SENSOR_ANGLES_DEG[sensor_id]
65     toward_speed = get_speed_toward_sensor(v_forward, v_right, sensor_angle)
66
67     if distance_cm < MIN_DISTANCE_CM and toward_speed > MIN_SPEED:
68         send_alert(vehicle)
69
70
71 def main():
72     vehicle = None
73
74     try:
75         vehicle = connect_pixhawk()
76
77         vehicle.message_factory.statustext_send(
78             mavutil.mavlink.MAV_SEVERITY_ALERT,
79             b"Companion computer connected"
80         )
81         vehicle.flush()
82
83         while True:
84             v_forward, v_right = get_body_velocity(vehicle)
85
86             d1 = sensor1.distance * 100 * MAX_DIST
87             check_distance(vehicle, d1, 1, v_forward, v_right)
88             time.sleep(GAP)
89
90             d2 = sensor2.distance * 100 * MAX_DIST
91             check_distance(vehicle, d2, 2, v_forward, v_right)
92             time.sleep(GAP)
93
94             d3 = sensor3.distance * 100 * MAX_DIST
95             check_distance(vehicle, d3, 3, v_forward, v_right)
96             time.sleep(GAP)
97
98     except KeyboardInterrupt:
99         pass
100
101     finally:
102         if vehicle:
103             vehicle.close()
104
105
```

```
106 if __name__ == "__main__":  
107     main()  
108
```

3.4 Prędkość jako jeden z warunków aktywacji

Następnym wykonanym programem, ukazanym w listingu 3.5 była wersja ostrzegająca pilota o przeszkodzie jedynie w sytuacji gdy dron zbliża się do niej z odpowiednio wysoką prędkością. Do zmiennych globalnych dodane zostały kąty pod jakimi ustawione są czujniki odległości oraz minimalna prędkość przy której program powinien zareagować. Oprócz wykonywania pomiarów program pobiera z komputera pokładowego informację o prędkości drona, która wyrażona jest w postaci trzech wartości, prędkości w kierunkach: północnym, wschodnim i w dół. Dzięki pobieranej z komputera pokładowego informacji o odchyleniu jednostki od kierunku północnego obliczana jest prędkość względem samego BSP oraz prędkości względem czujników. Mając informacje o względnej prędkości i odczytach z czujników odległości program porównuje je z progami określonymi w zmiennych globalnych i decyduje czy wywołać wysłanie ostrzeżenia.

Listing 3.4: Program ostrzegający o zbliżaniu się do przeszkody

```
1 import time  
2 import math  
3 from dronekit import connect  
4 from pymavlink import mavutil  
5  
6 from gpiozero import Device, DistanceSensor  
7 from gpiozero.pins.lgpio import LGPIOWFactory  
8  
9 Device.pin_factory = LGPIOWFactory()  
10  
11 DRONE_PORT = "/dev/serial/by-id/usb-Holybro_Pixhawk6C_450046001751333337363133-if00"   
12  
13 MIN_DISTANCE_CM = 30  
14 MIN_SPEED = 0.3 # w m/s  
15 GAP = 0.027 # 27ms  
16 MAX_DIST = 4.5  
17  
18 SENSOR_ANGLES_DEG = {  
19     1: -45.0,  
20     2: 0.0,  
21     3: 45.0  
22 }
```

```
23
24 sensor1 = DistanceSensor(echo=17, trigger=23, max_distance=MAX_DIST)
25 sensor2 = DistanceSensor(echo=27, trigger=24, max_distance=MAX_DIST)
26 sensor3 = DistanceSensor(echo=22, trigger=25, max_distance=MAX_DIST)
27
28
29 def connect_pixhawk():
30     while True:
31         try:
32             conn = mavutil.mavlink_connection(
33                 DRONE_PORT,
34                 baud=115200
35             )
36
37             conn.wait_heartbeat(timeout=60)
38             conn.close()
39
40             vehicle = connect(
41                 DRONE_PORT,
42                 baud=115200,
43                 wait_ready=True
44             )
45
46             return vehicle
47
48         except Exception:
49             time.sleep(3)
50
51
52 def get_body_velocity(vehicle):
53     velocity = vehicle.velocity
54     yaw = vehicle.attitude.yaw
55
56     if velocity is None or yaw is None:
57         return 0.0, 0.0
58
59     vn, ve, _ = velocity
60
61     v_forward = vn * math.cos(yaw) + ve * math.sin(yaw)
62     v_right = -vn * math.sin(yaw) + ve * math.cos(yaw)
63
64     return v_forward, v_right
65
66
67 def get_speed_toward_sensor(v_forward, v_right, sensor_angle_deg):
```

```
68     angle_rad = math.radians(sensor_angle_deg)
69     return v_forward * math.cos(angle_rad) + v_right * math.sin(angle_rad)
70
71
72 def send_alert(vehicle):
73     #[...]
74
75
76 def check_distance(vehicle, distance_cm, sensor_id, v_forward, v_right):
77     sensor_angle = SENSOR_ANGLES_DEG[sensor_id]
78     toward_speed = get_speed_toward_sensor(v_forward, v_right, sensor_angle)
79
80     if distance_cm < MIN_DISTANCE_CM and toward_speed > MIN_SPEED:
81         send_alert(vehicle)
82
83
84 def main():
85     vehicle = None
86
87     try:
88         vehicle = connect_pixhawk()
89
90         vehicle.message_factory.statustext_send(
91             mavutil.mavlink.MAV_SEVERITY_ALERT,
92             b"Companion computer connected"
93         )
94         vehicle.flush()
95
96         while True:
97             v_forward, v_right = get_body_velocity(vehicle)
98
99             d1 = sensor1.distance * 100 * MAX_DIST
100            check_distance(vehicle, d1, 1, v_forward, v_right)
101            time.sleep(GAP)
102
103            d2 = sensor2.distance * 100 * MAX_DIST
104            check_distance(vehicle, d2, 2, v_forward, v_right)
105            time.sleep(GAP)
106
107            d3 = sensor3.distance * 100 * MAX_DIST
108            check_distance(vehicle, d3, 3, v_forward, v_right)
109            time.sleep(GAP)
110
111     except KeyboardInterrupt:
112         pass
```

```

113
114     finally:
115         if vehicle:
116             vehicle.close()
117
118
119 if __name__ == "__main__":
120     main()
121

```

3.5 Dynamiczne ustawianie progu odległości

Poprzednie programy miały próg odległości określony w zmiennej globalnej. W tej wersji próg jest obliczany na bieżąco na podstawie prędkości drona oraz czasu potrzebnego na reakcję. Zamiast jednego progu aktywacji definiowanego z góry, Program odczytuje prędkość drona i ustala próg dynamicznie, dla każdego czujnika oddzielnie. W tym celu wykorzystano pomiary czasu reakcji i ustalono średni czas na 1.6 s. Na podstawie tych danych obliczana jest odległość, z jakiej pilot będzie w stanie zareagować na zbliżającą się przeszkodę.

Listing 3.5: Program ostrzegający o zbliżaniu się do przeszkody

```

1  import time
2  import math
3  import csv
4  from datetime import datetime
5
6  from dronekit import connect, APIException
7  from pymavlink import mavutil
8
9  from gpiozero import Device, DistanceSensor
10 from gpiozero.pins.lgpio import LGPIOFactory
11
12 Device.pin_factory = LGPIOFactory()
13
14 DRONE_PORT = "/dev/serial/by-id/usb-Holybro_Pixhawk6C_450046001751333337363133-if0
15
16 REACTION_TIME_S = 1.6
17 GAP = 0.027 # 27 ms
18 MAX_DIST_M = 4.5 # w metrach
19 MAX_DIST_CM = MAX_DIST_M * 100.0
20 LOG_FILE = "sensor_threshold_log.csv"
21
22 SENSOR_ANGLES_DEG = {

```

```
23     1: -45.0,
24     2: 0.0,
25     3: 45.0
26 }
27
28 sensor1 = DistanceSensor(echo=17, trigger=23, max_distance=MAX_DIST_M)
29 sensor2 = DistanceSensor(echo=27, trigger=24, max_distance=MAX_DIST_M)
30 sensor3 = DistanceSensor(echo=22, trigger=25, max_distance=MAX_DIST_M)
31
32
33 def init_log_file():
34     try:
35         with open(LOG_FILE, "x", newline="") as f:
36             writer = csv.writer(f)
37             writer.writerow([
38                 "timestamp",
39                 "sensor_id",
40                 "distance_cm",
41                 "toward_speed_m_s",
42                 "threshold_cm"
43             ])
44     except FileExistsError:
45         pass
46
47
48 def log_sensor_data(sensor_id, distance_cm, toward_speed, threshold_cm):
49     with open(LOG_FILE, "a", newline="") as f:
50         writer = csv.writer(f)
51         writer.writerow([
52             datetime.now().isoformat(timespec="seconds"),
53             sensor_id,
54             f"{distance_cm:.2f}",
55             f"{toward_speed:.2f}",
56             f"{threshold_cm:.2f}"
57         ])
58
59
60 def connect_pihawk():
61     # [...]
62
63
64 def get_body_velocity(vehicle):
65     # [...]
66
67
```

```
68 def get_speed_toward_sensor(v_forward, v_right, sensor_angle_deg):
69     #[...]
70
71
72 def calculate_threshold_cm(toward_speed):
73     closing_speed = max(0.0, toward_speed)
74
75     threshold_cm = closing_speed * REACTION_TIME_S * 100.0
76
77     threshold_cm = min(threshold_cm, MAX_DIST_CM)
78
79     return threshold_cm
80
81
82 def send_alert(vehicle, sensor_id):
83     #[...]
84
85
86 def check_distance(vehicle, distance_cm, sensor_id, v_forward, v_right):
87     sensor_angle = SENSOR_ANGLES_DEG[sensor_id]
88     toward_speed = get_speed_toward_sensor(v_forward, v_right, sensor_angle)
89     threshold_cm = calculate_threshold_cm(toward_speed)
90
91     log_sensor_data(sensor_id, distance_cm, toward_speed, threshold_cm)
92
93     print(
94         f"Sensor {sensor_id}: "
95         f"dystans={distance_cm:.2f} cm, "
96         f"prędkość={toward_speed:.2f} m/s, "
97         f"próg={threshold_cm:.2f} cm"
98     )
99
100     if threshold_cm > 0.0 and distance_cm < threshold_cm:
101         send_alert(vehicle, sensor_id, distance_cm, threshold_cm)
102
103
104 def main():
105     vehicle = None
106
107     try:
108         init_log_file()
109         vehicle = connect_pixhawk()
110
111         vehicle.message_factory.statustext_send(
112             mavutil.mavlink.MAV_SEVERITY_ALERT,
```



```
113         b"Companion computer connected"
114     )
115     vehicle.flush()
116
117     while True:
118         v_forward, v_right = get_body_velocity(vehicle)
119
120         d1 = sensor1.distance * 100.0
121         check_distance(vehicle, d1, 1, v_forward, v_right)
122         time.sleep(GAP)
123
124         d2 = sensor2.distance * 100.0
125         check_distance(vehicle, d2, 2, v_forward, v_right)
126         time.sleep(GAP)
127
128         d3 = sensor3.distance * 100.0
129         check_distance(vehicle, d3, 3, v_forward, v_right)
130         time.sleep(GAP)
131
132     except KeyboardInterrupt:
133         pass
134
135     finally:
136         if vehicle:
137             vehicle.close()
138
139
140 if __name__ == "__main__":
141     main()
142
```

3.6 Testy

W celu weryfikacji działania zbudowanego systemu wykonano szereg testów obejmujących zarówno sprawdzenie działania i parametrów użytego sprzętu, jak również poprawność działania napisanych programów.

3.6.1 Sprawdzenie poprawności działania systemu

Czujniki odległości zostały przetestowane poprzez ustawienie płaskiej przeszkody prostopadle do czujnika, w odległościach 30 cm, 50 cm, 1 m, 2 m i 4 m. We wszystkich przypadkach wykonane pomiary nie odbiegały od rzeczywistej odległości powyżej 1 cm różnicy, co jest wystarczającą dokładnością dla wykrywania przeszkód. W celu

sprawdzenia działania programu biorącego pod uwagę prędkość zbliżania się do przeszkody wykonano imitację przeszkody montowaną na BSP kilka centymetrów przed jednym z czujników odległości. Dron następnie był wznoszony w powietrze i rozpędzany do prędkości przekraczającej minimalną wymaganą do wysłania ostrzeżenia. Próbę powtórzono dla wszystkich trzech czujników.

Połączenie między komputerem pokładowym i asystującym jest weryfikowane poprzez wysłanie komunikatu o pomyślnym otwarciu połączenia. Komunikat ten może być odczytany przy użyciu komputera z aplikacją Mission Planner.

3.6.2 Pobór prądu

Pobór prądu Raspberry Pi w trakcie działania programu został zmierzony z użyciem miernika KWS-2302C. Pomiary wahały się pomiędzy 3.371 W a 3.612 W, w większości oscylując około 3.491 W. Biorąc pod uwagę parametry akumulatora zasilającego komputer asystujący można oszacować czas pracy na jednym ładowaniu na około 10.5 godziny.

3.6.3 Dokładność pomiarów odległości

Czujniki odległości HC-SR04, pomimo tego samego oznaczenia, w zależności od producenta mogą różnić się parametrami takimi jak maksymalna odległość wykonywania pomiarów. Użycie ultradźwięków wiąże się też z problemami z wykrywaniem przeszkód ustawionych pod kątem, fala dźwiękowa odbita od przeszkody może nigdy nie wrócić do czujnika. W związku z tym wykonano testy z użyciem czujników wykorzystanych w projekcie. Czujniki były w stanie zarejestrować przeszkodę ustawioną prostopadle z maksymalnie 460 cm, powyżej tej odległości odczyty były niestabilne i większość pomiarów kończyła się niewykryciem wracającej fali ultradźwiękowej. Sprawdzone również wpływ nachylenia przeszkody od pozycji prostopadłej do czujnika na poprawność odczytów. Pomiary wykonano w odległościach: 50 cm, 1 m, 2 m i 3 m. Pomiary wykazały:

- 50 cm— odczyty poprawne niezależnie od kąta odchylenia przeszkody,
- 1 m— odczyty niepoprawne przy odchyleniu $>50^\circ$,
- 2 m— odczyty niepoprawne przy odchyleniu $>22^\circ$,
- 3 m— odczyty niepoprawne przy odchyleniu $>12^\circ$.

3.6.4 Czas reakcji systemu

Przy wykrywaniu przeszkód przed poruszającym się pojazdem kluczowe jest odpowiednio niskie opóźnienie, tak aby osoba kierująca miała wystarczający czas na reakcję. W celu pomiaru opóźnień wykorzystano odpowiednio zmodyfikowany program `??`. Na potrzeby pomiaru zmieniono warunki aktywacji systemu tak, aby pomiary pierwszego i drugiego czujnika nie spełniały warunków potrzebnych do wysłania powiadomienia. Warunek dla trzeciego czujnika został zmodyfikowany tak by był on spełniany za każdym razem. Dzięki takim ustawieniom pomiary wykonywane były przy założeniu najbardziej pesymistycznego scenariusza, gdzie przeszkoda pojawia się w idealnym momencie by wymagała przejścia przez pozostałe czujniki zanim zostanie wykryta. Program rozpoczynał pomiar czasu po ustanowieniu połączenia z komputerem pokładowym i przed rozpoczęciem pomiarów odległości. W pierwszej wersji programu pomiar rozpoczynany był ręcznie poprzez naciśnięcie klawisza. Klawisz był następnie naciskany ponownie w momencie usłyszenia ostrzeżenia odczytanego przez komputer z aplikacją Mission Planner połączoną z dronem, co kończyło pomiar i zapisywało czas. W drugiej wersji programu ręczne uruchamianie pomiaru zostało zastąpione uruchamianiem automatycznym po losowym czasie 5–20 sekund. Wprowadzenie nieprzewidywalności do pomiaru pozwoliło na pomiar opóźnienia włącznie z ludzkim czasem reakcji, co w tym przypadku jest bardziej przydatne od opóźnień samego systemu. Wyniki obu pomiarów zostały przedstawione w tabeli [3.1](#)

Tabela 3.1: Pomiary czasu pomiędzy wykryciem przeszkody a reakcją użytkownika

Rozpoczynane ręcznie [s]	Rozpoczynane losowo [s]
1.078	1.655
0.905	1.399
6.770	1.493
1.339	1.388
1.014	1.525
0.862	1.289
0.862	1.400
0.775	1.482
0.884	1.380
0.710	1.519

3.7 Alternatywne rozwiązania

W trakcie pracy nad systemem zapobiegającym kolizjom powstały alternatywne rozwiązania i pomysły. Przykładowo program biorący pod uwagę prędkość drona przy decydowaniu o wysłaniu ostrzeżenia został napisany na podstawie wersji sprawdzającej jedynie odległość od przeszkody. W zależności od preferencji użytkownika uzależnienie ostrzeżeń od prędkości może być niepożądane.

Problem z różnicą napięć między stanem wysokim dla czujników odległości a stanem wysokim dla pinów GPIO w Raspberry Pi może być rozwiązany na inne sposoby niż z życiem konwertera stanów logicznych. Istnieje możliwość stworzenia dzielnika napięcia z pomocą dwóch odpowiednio dobranych rezystorów. Z powodu braku takich pod ręką ta możliwość nie została jednak przetestowana. Podczas prac czujniki odległości zostały podłączone do Arduino Nano. Wywoływanie pomiarów i obliczanie na ich podstawie zmierzonej odległości odbywało się na mikrokontrolerze, a wyniki przesyłane były do Raspberry Pi dzięki połączeniu USB. Takie rozwiązanie wprowadza jednak niepotrzebnie dodatkowe urządzenie do układu oraz zwiększa jego zużycie energii przez konieczność zasilenia mikrokontrolera. W przypadku gdy Raspberry Pi będzie działać z niedostatecznym zasilaniem może ograniczyć prąd idący do urządzeń połączonych przez USB do 600mA [6], w przypadku Arduino Nano takie ograniczenie nie stanowi jednak problemu ponieważ prąd wymagany do jego działania jest znacznie niższy. Użycie Raspberry Pi oraz Arduino komplikuje pracę nad oprogramowaniem; Do działającego systemu trzeba napisania dwóch programów w dwóch różnych językach, oddzielnie dla obu z urządzeń. Z powodu wymienionych wad użycia Arduino zostało ono zastąpione konwerterem stanów logicznych w późniejszych etapach rozwoju, pozostaje on jednak możliwym rozwiązaniem.

Użyte w tym przypadku ultradźwiękowe czujniki odległości nie są idealnymi urządzeniami pomiarowymi. Wykorzystanie fali dźwiękowej odbijającej się od przeszkody do określenia odległości wiąże się z ograniczeniami i podatnościami. Fala dźwiękowa, padająca na przeszkodę, odbija się różnie w zależności od jej materiału i kształtu. Dobrze wygłuszający materiał taki jak grube płótno czy gąbka może zostać niewykryty przez czujnik ultradźwiękowy. Czujniki wykorzystujące ultradźwięki mają również problem z wykrywaniem przeszkód ustawionych pod zbyt dużym kątem. Idealne warunki pomiarowe to przeszkoda ustawiona prostopadle do czujnika. Naukowcy Manabu Ishihara, Makoto Shiina i Shin-nosuke Suzuki przeprowadzili szereg eksperymentów z użyciem czujnika ultradźwiękowego [7] i odkryli że odległość od przeszkody ma wpływ na dopuszczalny kąt pomiaru. Wraz ze skracaniem dystansu można pozwolić sobie na coraz większy kąt i zachowanie niezawodności czujnika. Alternatywą dla czujników ultradźwiękowych mogą być moduły oparte o lasery podczerwone. Ponieważ wiązka

lasera porusza się z prędkością światła, znacznie szybciej niż prędkość fali dźwiękowej, pomiary laserem są o wiele szybsze od tych wykonywanych przez czujnik ultradźwiękowy. Używanie wielu czujników laserowych nie wiąże się też z ryzykiem zaburzania odczytów przez siebie nawzajem, co w praktyce eliminuje potrzebę ustawiania opóźnień między pomiarami. Z drugiej strony użycie lasera sprawia że obiekty szklane lub zbyt odbijające światło mogą nie zostać poprawnie wykryte. Czujniki wykorzystujące laser podczerwony są też zazwyczaj droższe niż czujniki ultradźwiękowe. Zastosowanie czujników HC-SR04 motywowane było głównie ich niską ceną oraz dostępnością, jednak wymienione wcześniej zalety czujników laserowych czynią z nich wartą przemyślenia alternatywę.

Rozdział 4

Potencjalne ścieżki rozwoju projektu

- 4.1 Półautonomiczny system zapobiegania kolizjom
- 4.2 Wielokierunkowy monitoring
- 4.3 Sieć neuronowa rozpoznawanie obiektów z obrazu kamery
- 4.4 mapa 3d lidar

Bibliografia

- [1] ArduPilot. Mission planner overview, 2026. dostę: 13.02.2026.
- [2] EASA. Drone regulatory system, 2026. dostę: 11.02.2026.
- [3] Holybro. Air2216ii, blheli s 20a, t1045ii documentation. Technical report, 2026. dostę: 13.02.2026.
- [4] Holybro. M10 gps module, 2026. dostę: 13.02.2026.
- [5] ExpressLRS LLC. Long range competition, 2022. dostę: 05.06.2026.
- [6] Raspberry Pi Ltd. Raspberry pi documentation, 2026. dostę: 26.04.2026.
- [7] Shin-nosuke Suzuki Manabu Ishihara, Makoto Shiina. Evaluation of method of measuring distance between object and walls using ultrasonic sensors. *Journal of Asian Electric Vehicles*, 7(1):1207–1211, June 2009.
- [8] PX4. Pixhawk 6c overview, 2026. dostę: 13.02.2026.
- [9] Cytron Technologies. Hc-sr04 user manual, 2013. dostę: 11.05.2026.